

SQL vs NoSQL

Complete Study Notes

Structure · Nature · Scalability · Properties
ACID vs BASE · NoSQL Types · When to Use Each

TOPICS COVERED

1. Four Key Dimensions Overview

2. SQL Structure & Nature

3. NoSQL Types (KV/Doc/Column/Graph)

4. ACID vs BASE Properties

5. Scalability — Vertical vs Horizontal

6. When to Use SQL vs NoSQL

7. Interview Decision Framework

8. Quick Revision & Interview Questions

Table of Contents

1. What is SQL? What is NoSQL?	3
2. Four Dimensions — Overview	3
3. Structure — SQL Tables vs NoSQL Models	3
4. NoSQL Types Deep Dive	4
5. Nature — Centralised vs Distributed	5
6. Scalability — Vertical vs Horizontal	5
7. Properties — ACID vs BASE	6
8. When to Use SQL vs NoSQL	7
9. Quick Revision & Interview Questions	7

1. What is SQL? What is NoSQL?

SQL (Structured Query Language) databases are **relational** — they store data in tables with rows and columns. Relationships between tables are enforced through foreign keys. They follow a **predetermined schema**: you must define the table structure before inserting any data.

NoSQL (Not Only SQL) databases encompass a broad family of non-relational data stores. "Not Only SQL" means they are not limited to the tabular model. They support multiple data structures — key-value pairs, documents, wide columns, and graphs — each optimised for different access patterns.

The name "NoSQL" is misleading — it doesn't mean SQL is never used. Many NoSQL databases now support SQL-like query languages.

Neither SQL nor NoSQL is universally "better" — the choice depends entirely on your data model, consistency requirements, and scale.

In interviews: never say "I'll use SQL" without justifying why — always connect the choice to your system's specific requirements.

2. Four Dimensions — Overview

SQL vs NoSQL — Four Key Dimensions

Structure	Nature	Scalability	Properties
SQL Tables · rows · columns Predetermined schema Relational dependencies	SQL Centralised Single server Sharding is hard	SQL Vertical (scale-up) Add CPU/RAM/storage Complex horizontal	SQL ACID Atomicity Consistency Isolation Durability
NoSQL Key-Value / Document Column-Family / Graph Flexible / dynamic schema	NoSQL Distributed Multiple nodes Naturally decentralised	NoSQL Horizontal (scale-out) Add more servers Elastic easy	NoSQL BASE Basically Available Eventual consistency

SQL: Relational · Centralised · Vertical · ACID

NoSQL: Multiple models · Distributed · Horizontal · BASE

Neither is better — choose based on your requirements

Four dimensions: Structure · Nature · Scalability · Properties — SQL top half, NoSQL bottom half in each card

3. Structure — SQL Tables vs NoSQL Models

SQL Structure

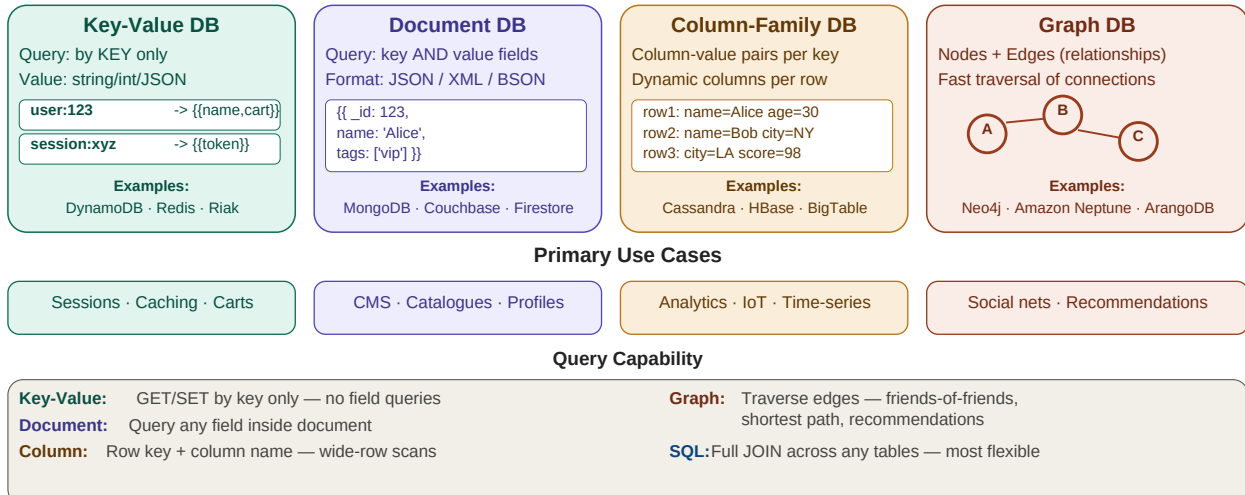
SQL databases use a **relational model**. Data lives in tables. Every table has a defined schema — column names and data types must be declared before any row is inserted. Tables relate to each other through **foreign keys**, enabling JOIN queries.

- Schema is **predetermined** — ALTER TABLE to change structure, which can be disruptive at scale
- Data integrity enforced at database level — NOT NULL, UNIQUE, FOREIGN KEY constraints
- Relationships: one-to-one, one-to-many, many-to-many — all expressed as joins
- Best when data has clear structure and relationships that don't change often

NoSQL Structure

NoSQL databases offer **schema flexibility**. Different records can have different fields. The structure adapts to what the application stores, not the other way around.

NoSQL Database Types — Models Examples



Four NoSQL models: Key-Value (query by key only) · Document (query any field) · Column-Family (dynamic columns per row) · Graph (nodes and edges)

4. NoSQL Types — Deep Dive

Type	Structure	Query by	Best for	Examples
Key-Value	key -> opaque value (string/JSON/binary)	Key only — no field queries	Sessions, caches, counters, shopping carts	DynamoDB, Redis, Riak, Memcached
Document	key -> structured document (JSON/BSON/XML)	Key + any field inside document	User profiles, catalogues, CMS, product listings	MongoDB, Couchbase, Firestore, CouchDB
Column-Family	row key -> dynamic column:value pairs	Row key + column name	Analytics, IoT, time-series, wide-row scans	Cassandra, HBase, Google BigTable
Graph	Nodes + Edges with properties	Traverse relationships (edges)	Social networks, recommendations, fraud detection	Neo4j, Amazon Neptune, ArangoDB, JanusGraph

Key-Value vs Document — Important Distinction

This is a common interview confusion. Both use a key, but they differ in how the value is treated:

- **Key-Value:** value is opaque to the database — you can only GET/SET by key. The DB doesn't know what's inside the value.
- **Document:** value is a structured document — the DB understands the fields inside. You can query WHERE age > 25 or find documents with a specific tag.
- Think of KV as a dictionary, Document as a searchable object store.

Column-Family — Why It's Special

- Traditional SQL: every row has the same columns. Empty cells waste storage.
- Column-Family: each row can have a completely different set of columns — sparse storage is efficient.
- Example: User 1 has {name, email}, User 2 has {name, phone, address} — perfectly fine in Cassandra.
- Excellent for write-heavy workloads — Cassandra writes are O(1) appends to a log.

Graph DB — When Relationships Are First-Class

- SQL can model relationships via JOINS, but performance degrades with depth (friends-of-friends = multiple joins).
- Graph DB: relationships (edges) are stored explicitly — traversal is O(1) per hop regardless of total data size.
- Use case: "find all users within 3 hops of Alice who bought product X" — trivial in Neo4j, expensive in SQL.

5. Nature — Centralised vs Distributed

The **nature** dimension describes how data is physically distributed across the infrastructure.

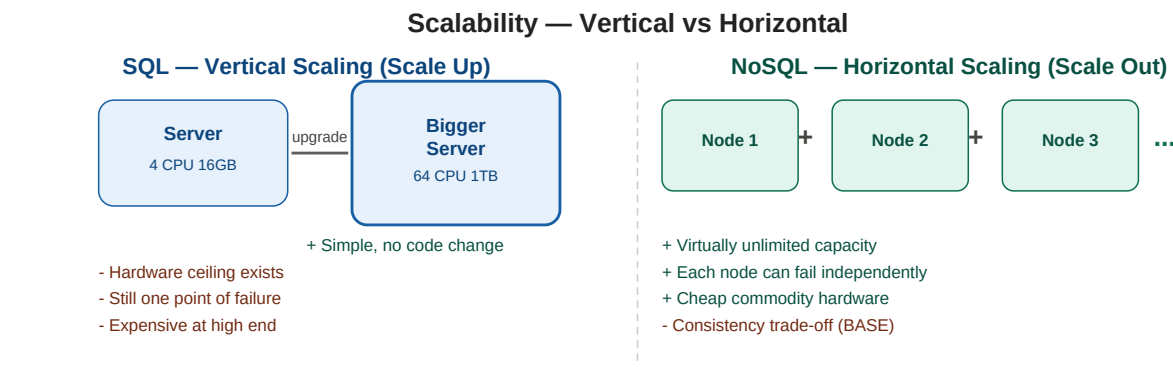
Aspect	SQL — Centralised	NoSQL — Distributed
Data location	Single server (primary)	Multiple nodes across cluster
Sharding support	Possible but complex (not native)	Built-in, automatic (Cassandra, DynamoDB)
Replication	Master-slave (manual config)	Auto-replicated (configurable N)
Failure impact	Primary down = system down	One node down = others continue
Network dependency	Low (all on one machine)	High (inter-node communication)
Data locality	All data together — easy joins	Data partitioned — cross-shard joins hard

SQL sharding IS possible: Vitess (MySQL), Citus (PostgreSQL), manual application-level sharding.

But NoSQL was designed distributed from day one — sharding is native and automatic.

Distributed nature of NoSQL means it can tolerate network partitions — hence the AP choice in CAP theorem.

6. Scalability — Vertical vs Horizontal



SQL sharding IS possible (e.g. Vitess for MySQL, Citus for PostgreSQL) but complex — not native like NoSQL
 NoSQL sharding IS native (MongoDB, Cassandra, DynamoDB handle it automatically)

SQL scales up (bigger server) · NoSQL scales out (more servers) · NoSQL sharding is native and automatic

7. Properties — ACID vs BASE

The properties dimension is often the deciding factor in database selection. ACID and BASE represent fundamentally different guarantees about how the system behaves during concurrent writes and failures.

ACID vs BASE — Properties Compared

ACID (SQL)	BASE (NoSQL)
A — Atomicity All or nothing — if any step fails, entire transaction rolls back	BA — Basically Available System guarantees availability — even if some nodes are down or stale
C — Consistency Data is always valid — constraints and rules are always enforced	S — Soft State State may change even without user input — background sync can update data
I — Isolation Concurrent transactions do not see each other's partial results	E — Eventual Consistency Replicas may diverge temporarily but converge to same value over time
D — Durability Committed data persists even after crash or power loss	Trade-off: High availability + partition tolerance at cost of immediate consistency

Use ACID when: data correctness is non-negotiable (banking, payments, inventory)

Use BASE when: availability and scale matter more than instant consistency (social feeds, caches, analytics)

ACID: all-or-nothing transactions, strong consistency · BASE: available and scalable, eventual consistency

ACID — Real-world Example

Bank transfer: debit \$100 from Account A, credit \$100 to Account B. Without ACID:

- Without Atomicity: debit succeeds, credit fails — \$100 disappears
- Without Isolation: another query reads Account A mid-transaction — sees wrong balance
- Without Durability: transfer commits, server crashes — transfer is lost
- ACID guarantees none of these happen — critical for financial systems

BASE — Real-world Example

Facebook like count: you like a post. Without BASE awareness:

- The like count on your screen may be 1,234 while your friend sees 1,233 — temporary divergence
- After a few seconds, all replicas sync — everyone sees 1,234 (eventual consistency)
- This is acceptable — nobody needs perfect real-time accuracy on a like counter
- The system stays highly available and fast — no coordination lock required

8. When to Use SQL vs NoSQL

When to Use SQL vs NoSQL — Decision Guide

Choose SQL when...	Choose NoSQL when...
Data has strong relationships Parent-child deps, foreign keys, joins across tables	Data is loosely related No strict FK dependencies — user events, logs, sessions
ACID compliance is required Banking, payments, inventory — data must be exact	High availability is paramount AP over CP — system stays up even during partitions
Complex, flexible queries Multi-table JOINS, GROUP BY, aggregations, reporting	Schema is flexible or unknown Structure evolves rapidly — startups, rapid iteration
Schema is stable and known Structure won't change often — ERP, CRM, legacy apps	Massive scale required Billions of records — horizontal sharding is automatic
Scale is moderate Vertical scaling covers your needs — millions not billions	Query patterns are simple GET by key, document lookup — social feeds, IoT, analytics
SQL examples: MySQL · PostgreSQL · Oracle · SQL Server · SQLite	NoSQL examples: MongoDB · Cassandra · DynamoDB · Redis · Neo4j · HBase

SQL: relationships, ACID, complex queries, stable schema · NoSQL: distributed, flexible, high availability, massive scale

Interview Decision Framework

When asked "SQL or NoSQL?" in a design interview, walk through these five questions:

Question	SQL answer	NoSQL answer
How related is the data?	Strong FK relationships, parent-child	Loosely coupled, independent records
What are query patterns?	Complex joins, aggregations, reports	Simple lookups by key/field
What consistency is needed?	Strong ACID — banking, payments	Eventual OK — social, analytics, IoT
What is the scale?	Millions of rows, vertical ok	Billions of records, need horizontal
Is schema stable?	Yes — ERP, legacy, financial	No — rapid iteration, evolving structure

9. Quick Revision & Interview Questions

Quick Revision

- > SQL = relational, tables, rows, columns, predetermined schema, foreign keys.
- > NoSQL = multiple models (KV, Document, Column-Family, Graph), flexible schema.
- > SQL nature: centralised — single server, sharding is complex (not native).
- > NoSQL nature: distributed — data partitioned across nodes, sharding is automatic.
- > SQL scalability: vertical (scale up — more CPU/RAM). Hits hardware ceiling.
- > NoSQL scalability: horizontal (scale out — add more nodes). Virtually unlimited.
- > ACID: Atomicity (all-or-nothing), Consistency, Isolation, Durability — SQL guarantee.
- > BASE: Basically Available, Soft state, Eventual Consistency — NoSQL trade-off.
- > Key-Value: query by KEY only. Value is opaque. Examples: DynamoDB, Redis.
- > Document DB: query by key AND fields inside document. Examples: MongoDB, Firestore.
- > Column-Family: dynamic columns per row — great for sparse wide-row data. Cassandra, HBase.

- > Graph DB: nodes + edges — fast traversal of relationships. Neo4j, Amazon Neptune.
- > Use SQL: banking, payments, ERP, complex reports, stable schema, strong consistency.
- > Use NoSQL: social media, IoT, caches, sessions, big data, flexible schema, high availability.
- > CAP: SQL favours CP (consistency + partition tolerance). NoSQL typically AP (availability + partition).

Interview Questions

- 1. What is the difference between SQL and NoSQL databases?
- 2. Explain the four types of NoSQL databases with examples.
- 3. What is the difference between a Key-Value store and a Document database?
- 4. What does ACID stand for? Give a real-world example where each property matters.
- 5. What does BASE stand for? When is eventual consistency acceptable?
- 6. Why is NoSQL better suited for horizontal scaling than SQL?
- 7. Can SQL databases scale horizontally? How?
- 8. When would you choose MongoDB over MySQL? Give a concrete scenario.
- 9. When would you choose MySQL over MongoDB? Give a concrete scenario.
- 10. What is a Column-Family database and when would you use it over a document DB?
- 11. When would you use a Graph database? Give an example.
- 12. Design a database for a social network. What would you choose and why?
- 13. Design a database for a banking system. What would you choose and why?

Key Terms

SQL	Structured Query Language — relational databases with tables, rows, schemas
NoSQL	Not Only SQL — non-relational databases: KV, Document, Column-Family, Graph
RDBMS	Relational Database Management System — MySQL, PostgreSQL, Oracle, SQL Server
Predetermined Schema	Table structure defined before any data is inserted — rigid but enforced
Foreign Key	Column referencing primary key of another table — enforces relationships
Key-Value Store	Database where each key maps to an opaque value — query by key only
Document DB	Stores structured documents (JSON/BSON) — queryable by any field
Column-Family	Rows with dynamic columns — efficient for sparse wide-column data
Graph DB	Nodes (entities) + Edges (relationships) — fast relationship traversal
ACID	Atomicity, Consistency, Isolation, Durability — strong transactional guarantees
BASE	Basically Available, Soft state, Eventual Consistency — NoSQL trade-off
Vertical Scaling	Scale up — add more CPU/RAM/storage to existing server
Horizontal Scaling	Scale out — add more servers/nodes to distribute load
Eventual Consistency	Replicas temporarily diverge but converge over time

CAP Theorem	Consistency, Availability, Partition Tolerance — choose 2 of 3
Sharding	Partitioning data across multiple nodes — native in NoSQL, complex in SQL